HireTrack Installer & Versioning Pipeline: Project Overview & Technical Architecture

Mission: To ship HireTrack as a versioned, pre-built server image and to install or upgrade it on a customer VM without compiling Next.js on that machine.

The Core Problem and The Solution

A production HireTrack build is large (on the order of hundreds of megabytes once `.next`, public assets, and lockfile are packed). Asking every customer to run `npm ci` and `next build` on a 1–2 GB VM fails: memory, time, and "works on my laptop." A single 600 MB download also fails on restrictive networks. In-app "update" cannot be a child of the Node process if the update must stop PM2 to free RAM.

The pipeline therefore builds once in CI, publishes a GitHub Release (full tarball plus 50 MB chunks and a SHA-256 manifest), and treats `installer.sh` as the only customer-facing tool. The installer is a small script the customer runs on the VM; the product bytes always come from Releases after License Admin has validated the machine. Source on `main` is not an installable build.

System Architecture & Release Workflow

1. **Version bump.** Two equivalent producers exist. Local `ci_prod.sh` (and `ci_dev.sh`) reads the latest GitHub tag, then bumps major / minor / patch from the last commit subject (`Major:`, `Feat:`, otherwise patch), writes `package.json` version, and continues. The GitHub Actions `Release` workflow uses the git tag `vX.Y.Z`, or on manual "Run workflow" uses `v` plus `package.json` version.
2. **Build.** `npm ci` with devDependencies, generate a runtime `.env`, `next build` with a raised Node heap, then delete `.next/cache` so customer disks are not filled with CI cache. `ci_prod.sh` also runs a production-only `npm ci` in a temp directory to prove the lockfile installs the way the installer will.
3. **Pack.** Tarball name `hiretrack-vX.Y.Z.tar.gz` contains runtime only: `package.json`, lockfile, `ecosystem.config.cjs`, `.next`, `.env`, `public`, `server-ws.cjs`. Not the Git tree, not `node_modules` (the installer runs production `npm ci` on the VM).
4. **Chunk.** `installer.sh --release-chunk-assets` splits the tarball into 50 MB parts and writes a manifest (`filename`, `parts`, `totalBytes`, `sha256`, part prefix). CI uploads the full archive, the manifest, and every `.part-NNN` to the GitHub Release. Those files are gitignored; they never live in the repo.
5. **Customer bootstrap.** The customer runs `installer.sh --install`. The script copies itself to `$HOME/.hiretrack/installer.sh` and re-execs from there. It registers the license with License Admin (email + machine code), then downloads the matching release. Optional `GITHUB_TOKEN` is used when the release repository is private.
6. **Download.** After a valid license, the installer resolves the latest tag (or a prerelease list for beta/staging channels). It prefers chunked parts: fetch manifest, download each part, concatenate, verify size and SHA-256, `gzip -t`. If chunks are missing, it falls back to the full tarball. A GitHub API `Not Found` means that version has no published Release yet.
7. **Host setup.** Interactive MongoDB Atlas vs local MongoDB, nginx + domain + TLS, Node, PM2 (`server-ws.cjs`), cron for auto-update and reminder pings, Tesseract for CV OCR, optional local markdown-parser. Config is `config.json` next to the app (installed version, previous version, channel, GitHub repo).

8. **Day-two.** `--update` compares installed tag to latest and replaces the tree. In-app update uses `GET /api/version/list` then `installer.sh --schedule-update` so the installer is not a child of Node. `--rollback` restores `previousVersion`. `--run-migrations from to` applies version-to-version Mongo scripts. Channels: production, beta, staging.

Target Audience Positioning

- **Maintainers** — run the Release workflow or `ci_prod.sh`; never tell customers to build from `main`.
- **Self-host operators** — one script for license registration, install, update, backup, domain, and rollback.
- **Restricted networks** — chunked parts exist so a 50 MB retry beats a failed 600 MB GET.

Technology Stack Summary

Domain	Technology	Implementation Purpose
Customer CLI	`installer.sh` (bash), jq, curl	Install, update, rollback, domain, cron, chunk helper.
Release CI	GitHub Actions `.github/workflows/release.yml`	Build, pack, chunk, and publish the GitHub Release on tag or workflow_dispatch.
Vendor CI (local)	`ci_prod.sh`, `ci_dev.sh`	Commit-message version bump, lockfile parity check, upload to GitHub Releases.
Artifact host	GitHub Releases	Source of truth for `hiretrack-vX.Y.Z.tar.gz` and parts. Not `main`.
In-app catalog	`GET /api/version/list`	Lists available Releases for a licensed install (same catalog License Admin exposes).
Runtime on VM	Node 20, PM2, nginx, MongoDB or Atlas	Serve `server-ws.cjs` behind TLS after `npm ci --omit=dev`.
Integrity	SHA-256 in manifest, gzip test	Detect truncated or swapped chunk downloads before extract.

Install and update require an active license. The tarball is served only after License Admin validates the machine. A missing GitHub Release for that version is a maintainer publishing problem, not something the customer can fix with an environment variable.